Ömer Faruk Akbay

22102584 HW1

1) $\arg\min_{W} \sum_{n} (y^n - h(x^n, W))^2 + \beta \sum_{i} w_i^2$

corresponds to $-\log p(W|D) = -\log p(D|W) - \log p(W)$

$\Rightarrow -\ln P(W) \propto \beta \sum_{i} w_i^2 \Rightarrow -\ln P(W) = \beta \sum_{i} w_i^2 + C$

$\Rightarrow \ln P(W) = -\beta \sum_{i} w_i^2 - C \Rightarrow P(W) = \exp\left(-\beta \sum_{i} w_i^2 - C\right)$

$\Rightarrow P(W) \propto \exp\left(-\beta \sum_{i=1}^{m} w_i^2\right)$

$\Rightarrow P(W) \propto \prod_{i=1}^{m} \exp(-\beta w_i^2)$, it is a Gaussian Dist.

$f(x) \propto \exp\left(-\frac{x^2}{2\sigma^2}\right) \Rightarrow \sigma^2 = \frac{1}{2\beta}$ so,

$\Rightarrow w_i \sim N\left(0, \frac{1}{2\beta}\right)$

Q 2)

2) a:

$$z = (x_1 \vee x_2') \oplus (x_3' \vee x_4')$$

$$= (x_1 \wedge x_3 \wedge x_4) \vee (x_2' \wedge x_3 \wedge x_4) \vee (x_1' \wedge x_2 \wedge x_3') \vee (x_1' \wedge x_2 \wedge x_4')$$

H_1: $w_{11} + w_{13} + w_{14} \geq \theta_1$, $w_{11} + w_{13} < \theta_1$, $w_{11} + w_{14} < \theta_1$, $w_{13} + w_{14} < \theta_1$

H_2: $w_{23} + w_{24} \geq \theta_2$, $w_{22} + w_{23} + w_{24} < \theta_2$, $w_{23} < \theta_2$, $w_{24} < \theta_2$

H_3: $w_{32} \geq \theta_3$, $w_{31} + w_{32} < \theta_3$, $w_{33} + w_{32} < \theta_3$, $0 < \theta_3$

H_4: $w_{42} \geq \theta_4$, $w_{41} + w_{42} < \theta_4$, $w_{44} + w_{42} < \theta_4$, $0 < \theta_4$

2.b)

We select the weights and biases to satisfy the inequalities.

H_1: W_1 = [1, 0, 1, 1], θ_1 = 2.3

H_2: W_2 = [0, -1, 1, 1], θ_2 = 1.2

H_3: W_3 = [-1, 1, -1, 0], θ_3 = 0.4

H_4: W_4 = [-1, 1, 0, -1], θ_4 = 0.6

2.c)

Robustness in a linear classifier is determined by its margin, which is the perpendicular distance from the closest data point to the decision boundary.

$$D = \frac{|W.X - \theta|}{||W||}$$

Since X_i is binary

For H_1, $||W|| = \sqrt{3}$,the Closest Positive Point is 3 and Closest Negative Point is 2.

So the optimal is

$$\theta_1 = \frac{3+2}{2} = 2.5$$

For H_2, $||W|| = \sqrt{3}$,the Closest Positive Point is 2 and Closest Negative Point is 1.

So the optimal is

$$\theta_2 = \frac{2+1}{2} = 1.5$$

For H_3, $||W|| = \sqrt{3}$,the Closest Positive Point is 1 and Closest Negative Point is 0.

So the optimal is

$$\theta_3 = \frac{1+0}{2} = 0.5$$

For H_4, $||W|| = \sqrt{3}$,the Closest Positive Point is 1 and Closest Negative Point is 0.

So the optimal is

$$\theta_4 = \frac{1+0}{2} = 0.5$$

2.d)

Network A (Unoptimized Thresholds) Accuracy: 85.0%

Network C (Robust Thresholds) Accuracy: 92.0%

For A: Since the decision boundaries are very close to actual data points, when there is a fluctuation in the data, the outcome is too effective.

For B: Since the decision boundaries are at an equal distance from the data points, the outcome is less affected.

Q 3)

Part A:

The Average Across-Class Correlation is **0.2240**, and the Average Within-Class Correlation is **0.3446** in Fig. 1. Since the within-class correlation is higher than the across-class correlation, this indicates low within-class variability. This is good because the model can better understand when samples from the same group look alike.

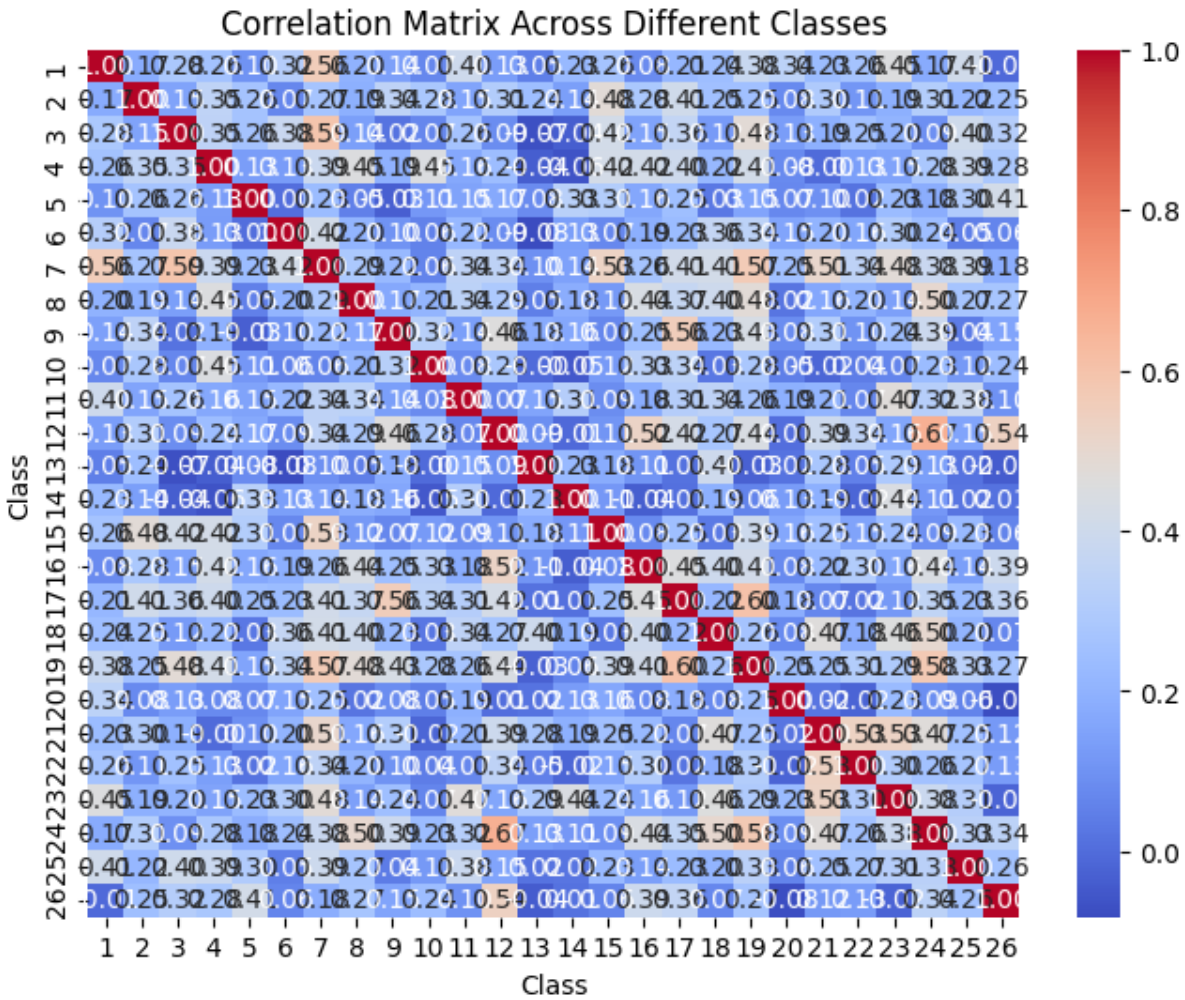

Figure 1

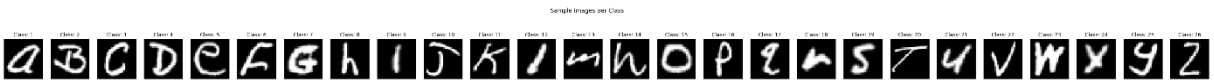

Figure 2

Part B:

A single-layer perceptron acts as a linear classifier; the weights for a specific output neuron become a template.

The red regions indicate positive weights that search for the digit's characteristic strokes in Fig. 3. The blue regions act as negative weights, suppressing the neuron's firing when pixels from competing digits are present. The MSE is optimal at 0.2, with a value of 0.0217.

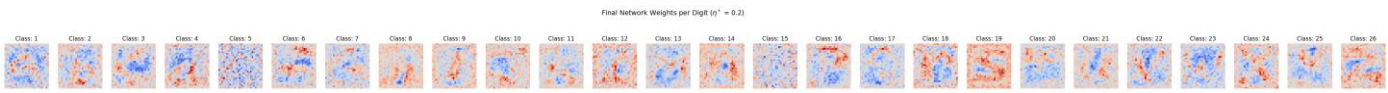

Figure 3

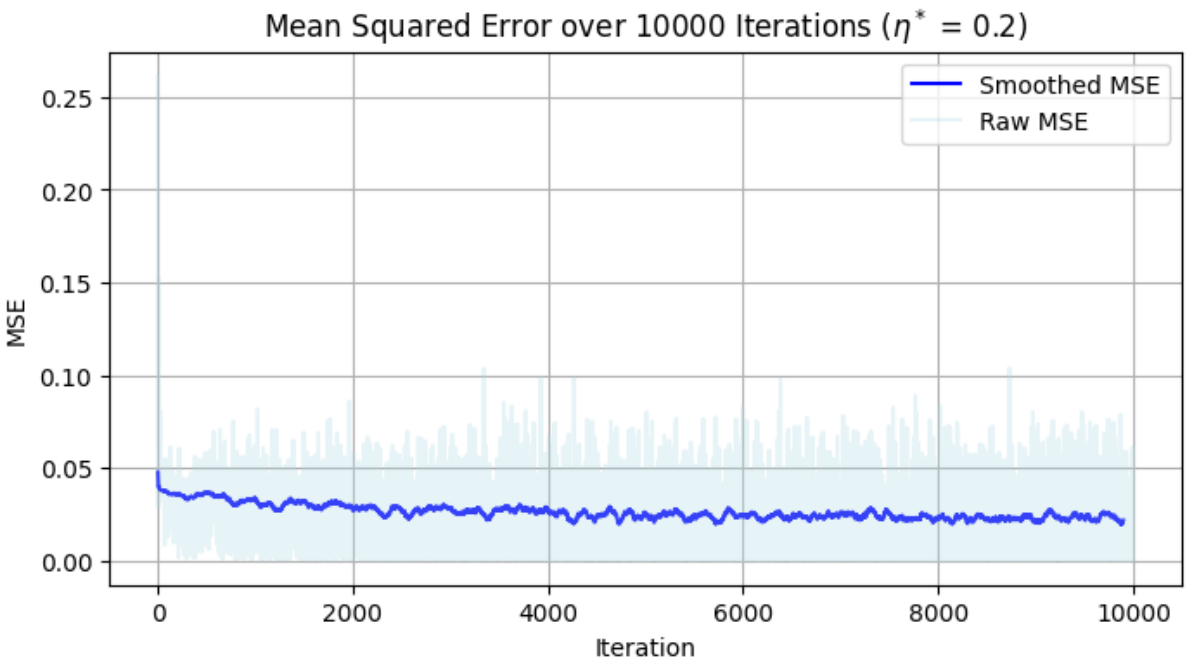

Figure 4

Part C:

In the Fig. 5;

Optimal Learning Rate: The green curve descends relatively quickly, then stabilizes at an MSE of 0.0217. It effectively reaches the plot's minimum.

High Learning Rate: Even if the red curve drops sharply, it shows unstable behavior and cannot descend after 0.3 MSE. This happens because the step size in gradient descent is too large, causing the algorithm to constantly overshoot the optimal weights.

Low Learning Rate: The blue curve reaches about 0.0288; however, because the step size is too small, it cannot reach the optimal value.

Training with optimal η* = 0.2...

Final smoothed MSE (η*): 0.0217

Training with high η_high = 2...

Final smoothed MSE (η_high): 0.0369

Training with low η_low = 0.02...

Final smoothed MSE (η_low): 0.0288

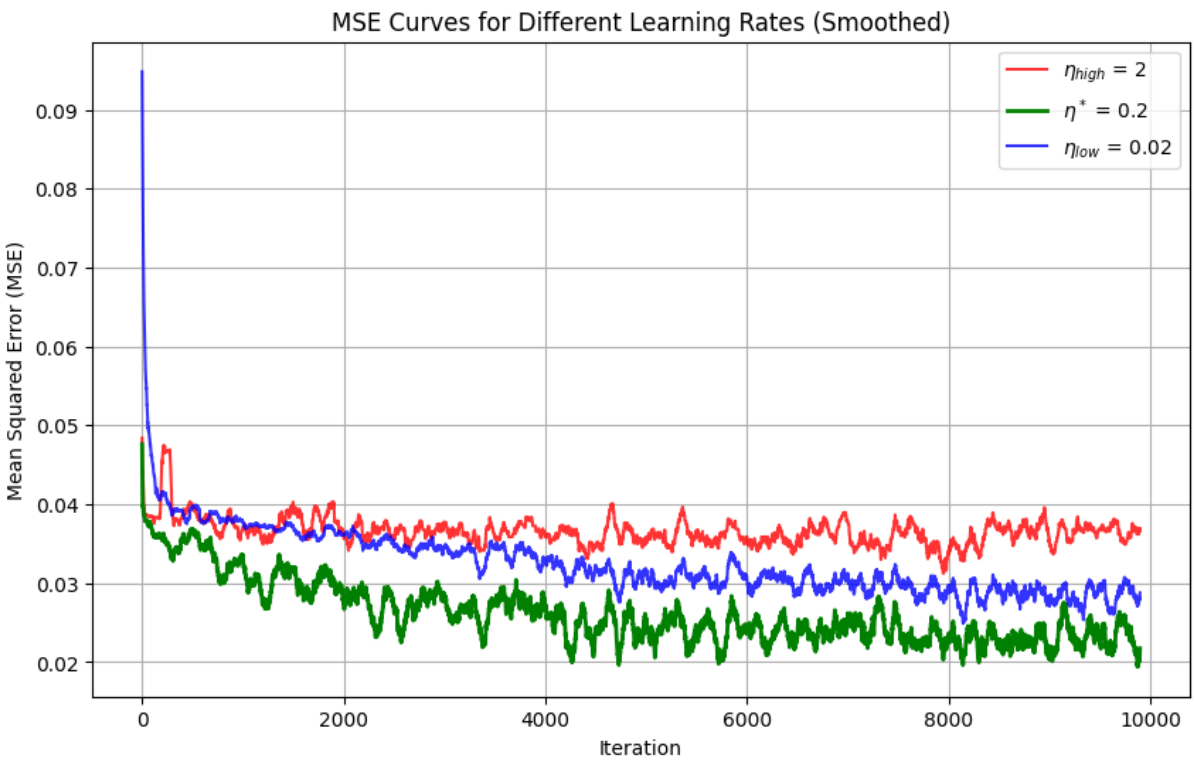

Figure 5

Part D:

--- Test Data Performance ---

Accuracy with optimal learning rate (η* = 0.2): 55.31%

Accuracy with high learning rate (η_high = 2): 15.15%

Accuracy with low learning rate (η_low = 0.02): 45.92%

As expected, the MSE values indicate that the optimal learning rate performs best. Also, with a high learning rate, performance is poor because the minimum is far away.

Q 4)

Final training loss: 0.017149607938732093

The loss is stabilized enough below 0.2

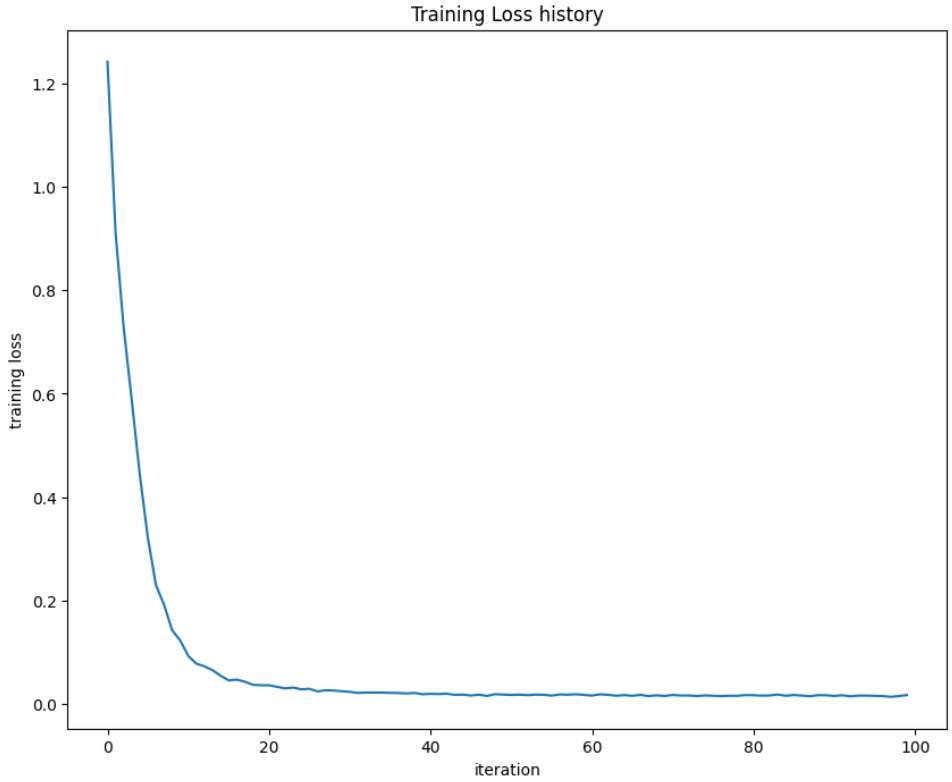

Figure 6

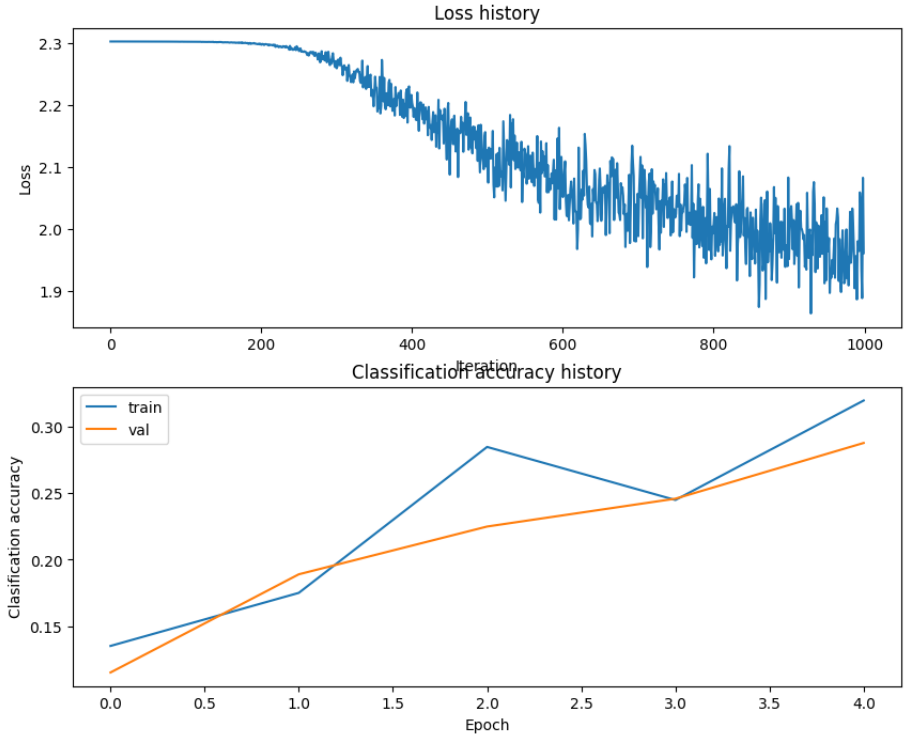

Figure 7

As shown in Fig. 7, the loss decreases linearly, so we can increase the learning rate. Also, we can increase model complexity since there is no overfitting yet.

The first training accuracy is Validation accuracy: 0.287. This is not very good; we need to improve it by arranging the hyperparameters.

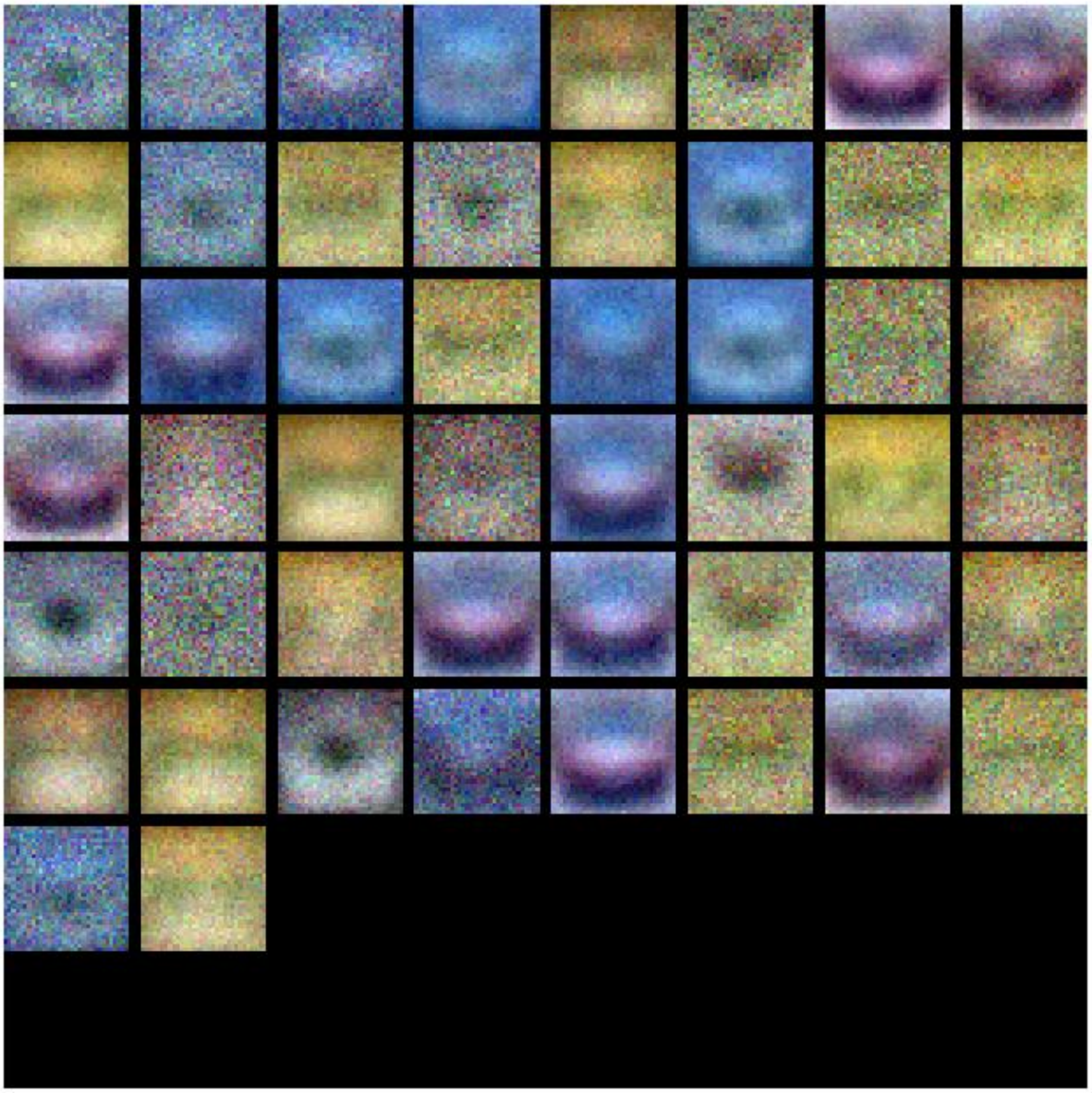

Figure 8

In Fig.8, you can see the initial weights before tuning. After the tuning, we get the best validation accuracy: 0.491000

Figure 9

In Fig.9, you can see the new weights that resemble the data.

Test accuracy: 0.482

Answer for Inline Question:
When there's a gap between test and training accuracy, the model is overfitting. To reduce this gap, we can train on **a larger dataset** to improve generalization and **increase regularization strength** to prevent the model from fitting noise in the training data. Adding more hidden units would increase model complexity and likely worsen overfitting.

REFERANCES

[1] Google, *Gemini* (Version 3.1 Pro) [Large Language Model]. Mountain View, CA: Google, 2026. Available: https://gemini.google.com

Description of Use: Assist in writing and debugging Python code using numpy and matplotlib for the single-layer perceptron training via online stochastic gradient descent.

Identify a vanishing gradient issue caused by unnormalized input data and suggest the standard pixel normalization fix.

APPENDİX

Q2:

```
"""

import numpy as np


# ==========================================

# 1. GENERATE THE DATASET

# ==========================================


# Define 4 base input vectors to represent our classes

# Positive cases (Target = 1):

pos_1 = [1, 0, 1, 1]  # Triggers Hidden Unit 1

pos_2 = [0, 1, 0, 1]  # Triggers Hidden Unit 3


# Negative cases (Target = 0):

# A=(1 OR NOT 1)=1, B=(NOT 0 OR NOT 0)=1 -> 1 XOR 1 = 0

neg_1 = [1, 1, 0, 0]

# A=(0 OR NOT 1)=0, B=(NOT 1 OR NOT 1)=0 -> 0 XOR 0 = 0

neg_2 = [0, 1, 1, 1]


base_inputs = np.array([pos_1, pos_2, neg_1, neg_2])
```

```
true_labels = np.array([1, 1, 0, 0])

# Concatenate 25 samples from each input vector
X_clean = np.repeat(base_inputs, 25, axis=0)
y_true = np.repeat(true_labels, 25)

# Generate random Gaussian noise (mean=0, std=0.2)
noise = np.random.normal(loc=0.0, scale=0.2, size=X_clean.shape)

# Form validation samples by superposing noise onto clean inputs
X_val = X_clean + noise

# ==========================================
# 2. DEFINE THE NEURAL NETWORKS
# ==========================================

# Weights derived in Part B (used for both networks)
W_hidden = np.array([
    [ 1,  0,  1,  1],  # H1
    [ 0, -1,  1,  1],  # H2
    [-1,  1, -1,  0],  # H3
    [-1,  1,  0, -1]   # H4
])

W_out = np.array([1, 1, 1, 1])
b_out = 0.5  # OR gate threshold

# --- Network A: Unoptimized (Fragile) Thresholds ---
```

```
b_hidden_part_a = np.array([2.9, 1.9, 0.9, 0.9])

# --- Network C: Most Robust Decision Boundary ---
b_hidden_part_c = np.array([2.5, 1.5, 0.5, 0.5])

def predict(X, w_h, b_h, w_o, b_o):
    #"""Forward pass through the 2-layer step-function network."""
    # Hidden Layer (Step function)
    hidden_activations = np.dot(X, w_h.T) >= b_h
    hidden_activations = hidden_activations.astype(int)

    # Output Layer (Step function)
    output_activations = np.dot(hidden_activations, w_o.T) >= b_o
    return output_activations.astype(int)

# ==========================================
# 3. EVALUATE CLASSIFICATION PERFORMANCE
# ==========================================

# Predictions
y_pred_A = predict(X_val, W_hidden, b_hidden_part_a, W_out, b_out)
y_pred_C = predict(X_val, W_hidden, b_hidden_part_c, W_out, b_out)

# Calculate Accuracy
accuracy_A = np.mean(y_pred_A == y_true) * 100
accuracy_C = np.mean(y_pred_C == y_true) * 100

print(f"Network A (Unoptimized Thresholds) Accuracy: {accuracy_A:.1f}%")
```

```
print(f"Network C (Robust Thresholds) Accuracy:      {accuracy_C:.1f}%")
"""
```

Q3:

```
# -*- coding: utf-8 -*-
"""Q3.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/130K2ceBUPN-oz1d5CljBPyypgJ5Xho_i
"""

#from google.colab import drive
#drive.mount('/content/drive')

#%cd /content/drive/MyDrive/neural/demo_tln

import h5py
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Load the dataset
file_name = 'assign1_data1.h5'
with h5py.File(file_name, 'r') as hf:
    trainims = np.array(hf['trainims'])
    trainlbls = np.array(hf['trainlbls']).flatten()
```

```
# Ensure standard shape (samples, features) or (samples, H, W)
# If dimensions are swapped (e.g., features, samples), transpose it
if trainims.shape[1] == trainlbls.shape[0] and len(trainims.shape) == 2:
    trainims = trainims.T

print(f"Data loaded successfully!")
print(f"Training images shape: {trainims.shape}")
print(f"Training labels shape: {trainlbls.shape}")

# 2. Select a sample image for each class
unique_classes = np.unique(trainlbls)
num_classes = len(unique_classes)
sample_images = []
sample_labels = []

for cls in unique_classes:
    # Get the index of the first occurrence of this class
    idx = np.where(trainlbls == cls)[0][0]
    sample_images.append(trainims[idx])
    sample_labels.append(int(cls))

sample_images = np.array(sample_images)

# 3. Determine image dimensions and visualize
# If the images are flattened 1D arrays, reshape them to 2D for plotting
if len(sample_images.shape) == 2:
    num_features = sample_images.shape[1]
    dim = int(np.sqrt(num_features))
```

```
    plot_images = sample_images.reshape(-1, dim, dim)
else:
    plot_images = sample_images

fig, axes = plt.subplots(1, num_classes, figsize=(num_classes * 2, 3))
for i, ax in enumerate(axes):
    # Depending on the dataset, you might need to transpose the image for correct viewing
    # e.g., ax.imshow(plot_images[i].T, cmap='gray') if it looks sideways
    ax.imshow(plot_images[i].T, cmap='gray')
    ax.set_title(f"Class: {sample_labels[i]}")
    ax.axis('off')
plt.suptitle("Sample Images per Class", y=1.05, fontsize=14)
plt.show()

# 4. Find correlation coefficients between the selected sample images
# Flatten the samples to 1D arrays for correlation calculation
flat_samples = sample_images.reshape(num_classes, -1)
correlation_matrix = np.corrcoef(flat_samples)

# 5. Display the correlations in a matrix format using Seaborn
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm',
            xticklabels=sample_labels, yticklabels=sample_labels)
plt.title("Correlation Matrix Across Different Classes")
plt.xlabel("Class")
plt.ylabel("Class")
plt.show()
```

```
# 6. Bonus: Calculate within-class correlation
within_class_corrs = []
for cls in unique_classes:
    indices = np.where(trainlbls == cls)[0]
    if len(indices) > 1:
        img1 = trainims[indices[0]].flatten()
        img2 = trainims[indices[1]].flatten()
        corr = np.corrcoef(img1, img2)[0, 1]
        within_class_corrs.append(corr)

avg_across_class = np.mean(correlation_matrix[np.triu_indices(num_classes, k=1)])
avg_within_class = np.mean(within_class_corrs)

print("\n--- Variability Discussion Metrics ---")
print(f"Average Across-Class Correlation (from matrix): {avg_across_class:.4f}")
print(f"Average Within-Class Correlation (first 2 samples of each class):
{avg_within_class:.4f}")

import numpy as np
import matplotlib.pyplot as plt

# 1. Setup Data and Labels
# --- THE FIX: Handle 3D shapes and flatten the images ---
if len(trainims.shape) == 3:
    # Check if the sample count is the last dimension (common in MATLAB)
    if trainims.shape[-1] == len(trainlbls):
        # Move the last axis to the front so it becomes (samples, H, W)
        trainims = np.moveaxis(trainims, -1, 0)
```

```
    num_samples = trainims.shape[0]
    num_features = trainims.shape[1] * trainims.shape[2]
    # Flatten the images so the perceptron can process them
    trainims_flat = trainims.reshape(num_samples, num_features)
    trainims_flat = trainims_flat / 255.0
else:
    num_samples, num_features = trainims.shape
    trainims_flat = trainims

unique_classes = np.unique(trainlbls)
num_classes = len(unique_classes)

# Create a mapping from actual label to index (0 to num_classes-1)
label_to_idx = {label: idx for idx, label in enumerate(unique_classes)}

# 2. Initialize Network Parameters
np.random.seed(42)
W = np.random.normal(0, np.sqrt(0.01), (num_classes, num_features))
b = np.random.normal(0, np.sqrt(0.01), (num_classes, 1))

# 3. Define Activation Function
def sigmoid(x):
    x = np.clip(x, -500, 500) # Clip to prevent overflow warnings
    return 1.0 / (1.0 + np.exp(-x))

def sigmoid_derivative(out):
    return out * (1.0 - out)
```

```
# 4. Training Setup
iterations = 10000
eta_star = 0.2 # Tuned learning rate
mse_history = []

W_train = W.copy()
b_train = b.copy()

# 5. Online Training Loop (Stochastic Gradient Descent)
for i in range(iterations):
    idx = np.random.randint(num_samples)

    # THE FIX: Ensure we use the flattened array here!
    x = trainims_flat[idx].reshape(-1, 1)

    target_class = label_to_idx[trainlbls[idx]]
    t = np.zeros((num_classes, 1))
    t[target_class, 0] = 1.0

    # --- Forward Pass ---
    z = np.dot(W_train, x) + b_train
    y = sigmoid(z)

    # Calculate MSE
    mse = np.mean((y - t)**2)
    mse_history.append(mse)
```

```
    # --- Backward Pass ---
    error = y - t
    delta = error * sigmoid_derivative(y)

    # Calculate gradients
    dW = np.dot(delta, x.T)
    db = delta

    # Update weights and biases
    W_train -= eta_star * dW
    b_train -= eta_star * db

print(f"Training completed. Final smoothed MSE: {np.mean(mse_history[-100:]):.4f}")

# 6. Visualize the Final Network Weights
dim = int(np.sqrt(num_features))
fig, axes = plt.subplots(1, num_classes, figsize=(num_classes * 2, 3))

for i, ax in enumerate(axes):
    weight_img = W_train[i, :].reshape(dim, dim)
    im = ax.imshow(weight_img.T, cmap='coolwarm')
    ax.set_title(f"Class: {int(unique_classes[i])}")
    ax.axis('off')

# THE FIX: Added 'r' before the string to make it a raw string for LaTeX
plt.suptitle(rf"Final Network Weights per Digit ($\eta^*$ = {eta_star})", y=1.05, fontsize=14)
plt.show()
```

```
# 7. Plot the MSE Curve
plt.figure(figsize=(8,4))
window_size = 100
smoothed_mse = np.convolve(mse_history, np.ones(window_size)/window_size,
mode='valid')
plt.plot(smoothed_mse, color='blue', label='Smoothed MSE')
plt.plot(mse_history, color='lightblue', alpha=0.3, label='Raw MSE')
# THE FIX: Added 'r' before the string here as well
plt.title(rf"Mean Squared Error over {iterations} Iterations ($\eta^*$ = {eta_star})")
plt.xlabel("Iteration")
plt.ylabel("MSE")
plt.legend()
plt.grid(True)
plt.show()

import numpy as np
import matplotlib.pyplot as plt

# 1. Define a reusable training function based on Part B
def train_perceptron(eta, iterations=10000):
    np.random.seed(42)
    W_train = np.random.normal(0, np.sqrt(0.01), (num_classes, num_features))
    b_train = np.random.normal(0, np.sqrt(0.01), (num_classes, 1))

    mse_history = []

    for i in range(iterations):
        idx = np.random.randint(num_samples)
```

```
        x = trainims_flat[idx].reshape(-1, 1)

        target_class = label_to_idx[trainlbls[idx]]
        t = np.zeros((num_classes, 1))
        t[target_class, 0] = 1.0

        # Forward Pass
        z = np.dot(W_train, x) + b_train
        y = sigmoid(z)

        # Calculate MSE
        mse = np.mean((y - t)**2)
        mse_history.append(mse)

        # Backward Pass
        error = y - t
        delta = error * sigmoid_derivative(y)

        dW = np.dot(delta, x.T)
        db = delta

        # Update weights
        W_train -= eta * dW
        b_train -= eta * db

    return mse_history

# 2. Define the Learning Rates
```

```
eta_star = 0.2
eta_high = 2
eta_low = 0.02

print(f"--- Training with different learning rates ---")

print(f"\nTraining with optimal \u03B7* = {eta_star}...")
mse_star = train_perceptron(eta_star)
print(f"Final smoothed MSE (\u03B7*): {np.mean(mse_star[-100:]):.4f}")

print(f"\nTraining with high \u03B7_high = {eta_high}...")
mse_high = train_perceptron(eta_high)
print(f"Final smoothed MSE (\u03B7_high): {np.mean(mse_high[-100:]):.4f}")

print(f"\nTraining with low \u03B7_low = {eta_low}...")
mse_low = train_perceptron(eta_low)
print(f"Final smoothed MSE (\u03B7_low): {np.mean(mse_low[-100:]):.4f}")

print("\nGenerating plot...")

# 3. Plotting the results on a single figure
plt.figure(figsize=(10, 6))

def smooth_curve(points, factor=100):
    return np.convolve(points, np.ones(factor)/factor, mode='valid')

plt.plot(smooth_curve(mse_high), label=rf'$\eta_{{high}}$ = {eta_high}', color='red',
alpha=0.8)
```

```
plt.plot(smooth_curve(mse_star), label=rf'$\eta^*$ = {eta_star}', color='green', linewidth=2)
plt.plot(smooth_curve(mse_low), label=rf'$\eta_{{low}}$ = {eta_low}', color='blue',
alpha=0.8)

plt.title("MSE Curves for Different Learning Rates (Smoothed)")
plt.xlabel("Iteration")
plt.ylabel("Mean Squared Error (MSE)")
plt.legend()
plt.grid(True)
plt.show()

import h5py
import numpy as np

# 1. Load the Test Data
file_name = 'assign1_data1.h5'
with h5py.File(file_name, 'r') as hf:
    testims = np.array(hf['testims'])
    testlbls = np.array(hf['testlbls']).flatten()

# Preprocess test images identically to training images
if len(testims.shape) == 3:
    if testims.shape[-1] == len(testlbls):
        testims = np.moveaxis(testims, -1, 0)

    num_test_samples = testims.shape[0]
    num_features = testims.shape[1] * testims.shape[2]
    testims_flat = testims.reshape(num_test_samples, num_features)
```

```
    testims_flat = testims_flat / 255.0
else:
    # If it's a 2D array and needs transposing
    if testims.shape[1] == len(testlbls):
        testims = testims.T
    testims_flat = testims
    num_test_samples = testims.shape[0]

# 2. Modify training function to return the final Weights and Biases
def get_trained_weights(eta, iterations=10000):
    np.random.seed(42)
    W_train = np.random.normal(0, np.sqrt(0.01), (num_classes, num_features))
    b_train = np.random.normal(0, np.sqrt(0.01), (num_classes, 1))

    for i in range(iterations):
        idx = np.random.randint(num_samples) # Using train_samples from Part B
        x = trainims_flat[idx].reshape(-1, 1)

        target_class = label_to_idx[trainlbls[idx]]
        t = np.zeros((num_classes, 1))
        t[target_class, 0] = 1.0

        # Forward & Backward Pass
        z = np.dot(W_train, x) + b_train
        y = sigmoid(z)
        error = y - t
        delta = error * sigmoid_derivative(y)
```

```
        W_train -= eta * np.dot(delta, x.T)
        b_train -= eta * delta

    return W_train, b_train

print("Training networks to extract final weights...")
W_star, b_star = get_trained_weights(eta_star)
W_high, b_high = get_trained_weights(eta_high)
W_low, b_low = get_trained_weights(eta_low)

# 3. Define the Evaluation Function
def evaluate_network(W, b, X_test, y_test_labels):
    # Vectorized forward pass for all test samples at once
    # X_test is (samples, features), W is (classes, features)
    # Resulting z will be (samples, classes)
    z = np.dot(X_test, W.T) + b.T
    y_pred = sigmoid(z)

    # The predicted class is the neuron with the highest activation
    pred_indices = np.argmax(y_pred, axis=1)

    # Map indices back to actual labels (e.g., index 0 -> label '1')
    pred_labels = unique_classes[pred_indices]

    # Calculate Accuracy
    correct = np.sum(pred_labels == y_test_labels)
    accuracy = (correct / len(y_test_labels)) * 100
    return accuracy
```

```
# 4. Report Performance

print("\n--- Test Data Performance ---")

print(f"Accuracy with optimal learning rate (\u03B7* =
{eta_star}):     {evaluate_network(W_star, b_star, testims_flat, testlbls):.2f}%")

print(f"Accuracy with high learning rate (\u03B7_high =
{eta_high}):   {evaluate_network(W_high, b_high, testims_flat, testlbls):.2f}%")

print(f"Accuracy with low learning rate (\u03B7_low =
{eta_low}):    {evaluate_network(W_low, b_low, testims_flat, testlbls):.2f}%")
```

Q4:

```
# -*- coding: utf-8 -*-

"""two_layer_net.ipynb


Automatically generated by Colab.


Original file is located at

    https://colab.research.google.com/drive/14Hcg62KtkzGdCWPw---YcPzAEd06d_SG


# Implementing a Neural Network

In this exercise we will develop a neural network with fully-connected layers to perform
classification, and test it out on the CIFAR-10 dataset.

"""


#from google.colab import drive

#drive.mount('/content/drive')


# Commented out IPython magic to ensure Python compatibility.

# %cd /content/drive/MyDrive/neural/demo_tln
```

```
# Commented out IPython magic to ensure Python compatibility.
# A bit of setup
from __future__ import print_function
import numpy as np
import matplotlib.pyplot as plt

from cs231n.classifiers.neural_net import TwoLayerNet


# %matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
#%load_ext autoreload
#%autoreload 2

def rel_error(x, y):
    """ returns relative error """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))

"""We will use the class `TwoLayerNet` in the file `cs231n/classifiers/neural_net.py` to
represent instances of our network. The network parameters are stored in the instance
variable `self.params` where keys are string parameter names and values are numpy arrays.
Below, we initialize toy data and a toy model that we will use to develop your
implementation."""
```

```
# Create a small net and some toy data to check your implementations.
# Note that we set the random seed for repeatable experiments.

input_size = 4
hidden_size = 10
num_classes = 3
num_inputs = 5

def init_toy_model():
    np.random.seed(0)
    return TwoLayerNet(input_size, hidden_size, num_classes, std=1e-1)

def init_toy_data():
    np.random.seed(1)
    X = 10 * np.random.randn(num_inputs, input_size)
    y = np.array([0, 1, 2, 2, 1])
    return X, y

net = init_toy_model()
X, y = init_toy_data()

"""# Forward pass: compute scores
```

Open the file `cs231n/classifiers/neural_net.py` and look at the method `TwoLayerNet.loss`. This function is very similar to the loss functions you have written for the SVM and Softmax exercises: It takes the data and weights and computes the class scores, the loss, and the gradients on the parameters.

Implement the first part of the forward pass which uses the weights and biases to compute the scores for all inputs.

"""

```
scores = net.loss(X)

print('Your scores:')

print(scores)

print()

print('correct scores:')

correct_scores = np.asarray([

  [-0.81233741, -1.27654624, -0.70335995],

  [-0.17129677, -1.18803311, -0.47310444],

  [-0.51590475, -1.01354314, -0.8504215 ],

  [-0.15419291, -0.48629638, -0.52901952],

  [-0.00618733, -0.12435261, -0.15226949]])

print(correct_scores)

print()

# The difference should be very small. We get < 1e-7

print('Difference between your scores and correct scores:')

print(np.sum(np.abs(scores - correct_scores)))
```

"""# Forward pass: compute loss

In the same function, implement the second part that computes the data and regularizaion loss.

"""

```
loss, _ = net.loss(X, y, reg=0.05)

correct_loss = 1.30378789133
```

```
# should be very small, we get < 1e-12
print('Difference between your loss and correct loss:')
print(np.sum(np.abs(loss - correct_loss)))

"""# Backward pass
Implement the rest of the function. This will compute the gradient of the loss with respect to the variables `W1`, `b1`, `W2`, and `b2`. Now that you (hopefully!) have a correctly implemented forward pass, you can debug your backward pass using a numeric gradient check:
"""

from cs231n.gradient_check import eval_numerical_gradient

# Use numeric gradient checking to check your implementation of the backward pass.
# If your implementation is correct, the difference between the numeric and
# analytic gradients should be less than 1e-8 for each of W1, W2, b1, and b2.

loss, grads = net.loss(X, y, reg=0.05)

# these should all be less than 1e-8 or so
for param_name in grads:
    f = lambda W: net.loss(X, y, reg=0.05)[0]
    param_grad_num = eval_numerical_gradient(f, net.params[param_name], verbose=False)
    print('%s max relative error: %e' % (param_name, rel_error(param_grad_num, grads[param_name])))

"""# Train the network
To train the network we will use stochastic gradient descent (SGD), similar to the SVM and Softmax classifiers. Look at the function `TwoLayerNet.train` and fill in the missing sections to
```

implement the training procedure. This should be very similar to the training procedure you used for the SVM and Softmax classifiers. You will also have to implement `TwoLayerNet.predict`, as the training process periodically performs prediction to keep track of accuracy over time while the network trains.

Once you have implemented the method, run the code below to train a two-layer network on toy data. You should achieve a training loss less than 0.2.

"""

```
net = init_toy_model()
stats = net.train(X, y, X, y,
            learning_rate=1e-1, reg=5e-6,
            num_iters=100, verbose=False)

print('Final training loss: ', stats['loss_history'][-1])

# plot the loss history
plt.plot(stats['loss_history'])
plt.xlabel('iteration')
plt.ylabel('training loss')
plt.title('Training Loss history')
plt.show()
```

"""# Load the data

Now that you have implemented a two-layer network that passes gradient checks and works on toy data, it's time to load up our favorite CIFAR-10 data so we can use it to train a classifier on a real dataset.

"""

"""import os

```
os.chdir('/content/drive/MyDrive/neural/demo_tln')
print("Now in:", os.getcwd())"""

import pathlib

p = pathlib.Path("cs231n/data_utils.py")
txt = p.read_text()

txt = txt.replace("from scipy.misc import imread",
                  "import imageio.v2 as imageio\nimread = imageio.imread")

p.write_text(txt)

print("Patched correctly ")

from cs231n.data_utils import load_CIFAR10

def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000):
    """
    Load the CIFAR-10 dataset from disk and perform preprocessing to prepare
    it for the two-layer neural net classifier. These are the same steps as
    we used for the SVM, but condensed to a single function.
    """
    # Load the raw CIFAR-10 data
    cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)
```

```
    # Subsample the data
    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]
    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]
    mask = list(range(num_test))
    X_test = X_test[mask]
    y_test = y_test[mask]

    # Normalize the data: subtract the mean image
    mean_image = np.mean(X_train, axis=0)
    X_train -= mean_image
    X_val -= mean_image
    X_test -= mean_image

    # Reshape data to rows
    X_train = X_train.reshape(num_training, -1)
    X_val = X_val.reshape(num_validation, -1)
    X_test = X_test.reshape(num_test, -1)

    return X_train, y_train, X_val, y_val, X_test, y_test

# Cleaning up variables to prevent loading data multiple times (which may cause memory
issue)
try:
   del X_train, y_train
```

```
  del X_test, y_test
  print('Clear previously loaded data.')
except:
  pass

# Invoke the above function to get our data.
X_train, y_train, X_val, y_val, X_test, y_test = get_CIFAR10_data()
print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)

"""# Train a network
To train our network we will use SGD. In addition, we will adjust the learning rate with an exponential learning rate schedule as optimization proceeds; after each epoch, we will reduce the learning rate by multiplying it by a decay rate.
"""

input_size = 32 * 32 * 3
hidden_size = 50
num_classes = 10
net = TwoLayerNet(input_size, hidden_size, num_classes)

# Train the network
stats = net.train(X_train, y_train, X_val, y_val,
          num_iters=1000, batch_size=200,
          learning_rate=1e-4, learning_rate_decay=0.95,
```

```
            reg=0.25, verbose=True)

# Predict on the validation set
val_acc = (net.predict(X_val) == y_val).mean()
print('Validation accuracy: ', val_acc)

"""# Debug the training
With the default parameters we provided above, you should get a validation accuracy of
about 0.29 on the validation set. This isn't very good.

One strategy for getting insight into what's wrong is to plot the loss function and the
accuracies on the training and validation sets during optimization.

Another strategy is to visualize the weights that were learned in the first layer of the
network. In most neural networks trained on visual data, the first layer weights typically
show some visible structure when visualized.
"""

# Plot the loss function and train / validation accuracies
plt.subplot(2, 1, 1)
plt.plot(stats['loss_history'])
plt.title('Loss history')
plt.xlabel('Iteration')
plt.ylabel('Loss')

plt.subplot(2, 1, 2)
plt.plot(stats['train_acc_history'], label='train')
plt.plot(stats['val_acc_history'], label='val')
plt.title('Classification accuracy history')
```

```
plt.xlabel('Epoch')
plt.ylabel('Clasification accuracy')
plt.legend()
plt.show()

from cs231n.vis_utils import visualize_grid

# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(32, 32, 3, -1).transpose(3, 0, 1, 2)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(net)

"""# Tune your hyperparameters
```

What's wrong?. Looking at the visualizations above, we see that the loss is decreasing more or less linearly, which seems to suggest that the learning rate may be too low. Moreover, there is no gap between the training and validation accuracy, suggesting that the model we used has low capacity, and that we should increase its size. On the other hand, with a very large model we would expect to see more overfitting, which would manifest itself as a very large gap between the training and validation accuracy.

Tuning. Tuning the hyperparameters and developing intuition for how they affect the final performance is a large part of using Neural Networks, so we want you to get a lot of practice. Below, you should experiment with different values of the various hyperparameters, including hidden layer size, learning rate, numer of training epochs, and regularization

strength. You might also consider tuning the learning rate decay, but you should be able to get good performance using the default value.

Approximate results. You should be aim to achieve a classification accuracy of greater than 48% on the validation set. Our best network gets over 52% on the validation set.

Experiment: You goal in this exercise is to get as good of a result on CIFAR-10 as you can, with a fully-connected Neural Network. Feel free implement your own techniques (e.g. PCA to reduce dimensionality, or adding dropout, or adding features to the solver, etc.).

"""

best_net = None # store the best model into this

```
################################################################################
# TODO: Tune hyperparameters using the validation set. Store your best trained  #
# model in best_net.                                                             #
#                                                                                #
# To help debug your network, it may help to use visualizations similar to the  #
# ones we used above; these visualizations will have significant qualitative    #
# differences from the ones we saw above for the poorly tuned network.          #
#                                                                                #
# Tweaking hyperparameters by hand can be fun, but you might find it useful to  #
# write code to sweep through possible combinations of hyperparameters          #
# automatically like we did on the previous exercises.                          #
################################################################################
```

```
input_size = X_train.shape[1]
hidden_size = 100
```

```
output_size = 10

# learning_rates = [1, 1e-1, 1e-2, 1e-3]
# regularization_strengths = [1e-6, 1e-5, 1e-4, 1e-3]

# Magic lrs and regs?
# Just copy from:
https://github.com/lightaime/cs231n/blob/master/assignment1/two_layer_net.ipynb
# :(
learning_rates = np.array([0.7, 0.8, 0.9, 1, 1.1])*1e-3
regularization_strengths = [0.75, 1, 1.25]

best_val = -1

for lr in learning_rates:
    for reg in regularization_strengths:
        net = TwoLayerNet(input_size, hidden_size, output_size)
        net.train(X_train, y_train, X_val, y_val, learning_rate=lr, reg=reg,
                  num_iters=1500)

        y_val_pred = net.predict(X_val)
        val_acc = np.mean(y_val_pred == y_val)

        print('lr: %f, reg: %f, val_acc: %f' % (lr, reg, val_acc))

        if val_acc > best_val:
            best_val = val_acc
            best_net = net
```

```
print('Best validation accuracy: %f' % best_val)

################################################################################
######
#                          END OF YOUR CODE                                    #
################################################################################
######


# visualize the weights of the best network

show_net_weights(best_net)


"""# Run on the test set

When you are done experimenting, you should evaluate your final trained network on the
test set; you should get above 48%.

"""


test_acc = (best_net.predict(X_test) == y_test).mean()

print('Test accuracy: ', test_acc)


"""**Inline Question**


Now that you have trained a Neural Network classifier, you may find that your testing
accuracy is much lower than the training accuracy. In what ways can we decrease this gap?
Select all that apply.

1. Train on a larger dataset.

2. Add more hidden units.

3. Increase the regularization strength.

4. None of the above.


*Your answer*:
```

Your explanation:

"""